Informe Técnico: Taller de Patrones de Diseño (Strategy y State)

Asignatura: Programación III
Docente: Alberto Paternina León
Grupo: 6
Integrantes: Juan Camilo Ibarra Mercado
Esteban González Palencia
Sebastián Fuentes Vidal

---

1. Introducción y Dominio de Aplicación

El presente informe documenta el desarrollo del taller de patrones de diseño correspondiente al par contrastante **Strategy + State** [cite: 1]. Para cumplir con la restricción de dominio establecida en la rúbrica (evitando los dominios canónicos de tutoriales como *Pizza*, *Animal* o *Shape*) [cite: 2], se ha modelado el sistema de gestión de un **Parqueadero Universitario**.

Este dominio permite evidenciar de forma natural la necesidad de ambos patrones: por un lado, el cálculo de tarifas varía según el tipo de usuario (estudiante, profesor, visitante), lo que se ajusta perfectamente a la selección de algoritmos del patrón *Strategy*; por otro lado, un espacio de parqueo experimenta un ciclo de vida con estados dinámicos (disponible, ocupado, en mantenimiento) y transiciones condicionadas, escenario idóneo para el patrón *State*.

---

2. Análisis del Contraste entre Strategy y State

Aunque estructuralmente ambos patrones comparten una similitud evidente —emplean la composición de objetos delegando la ejecución a través de una interfaz común—, sus intenciones de diseño y propósitos fundamentales divergen por completo [cite: 1]:

Dimensión de Análisis	**Patrón Strategy**	**Patrón State**
Intención Principal	Permitir la selección e intercambio de un **algoritmo**	Permitir que un objeto **altere su comportamiento interno**

Dimensión de Análisis	Patrón Strategy	Patrón State
	o familia de comportamientos en tiempo de ejecución de manera transparente para el cliente [cite: 1].	cuando su estado cambia, aparentando cambiar de clase [cite: 1].
Conocimiento del Contexto	Las estrategias concretas por lo general son independientes y pasivas; reciben los datos necesarios como parámetros o consultan al contexto de forma desacoplada.	Los estados concretos suelen conocer íntimamente al contexto y, con frecuencia, son responsables de **disparar y gestionar las transiciones** hacia otros estados.
Cambio de Comportamiento	Es impulsado de manera **externa** (el cliente o la aplicación decide explícitamente qué estrategia instanciar o asignar según los parámetros de entrada).	Es impulsado de manera **interna** (el propio flujo operativo del sistema o los eventos sobre el objeto provocan la evolución de un estado a otro).

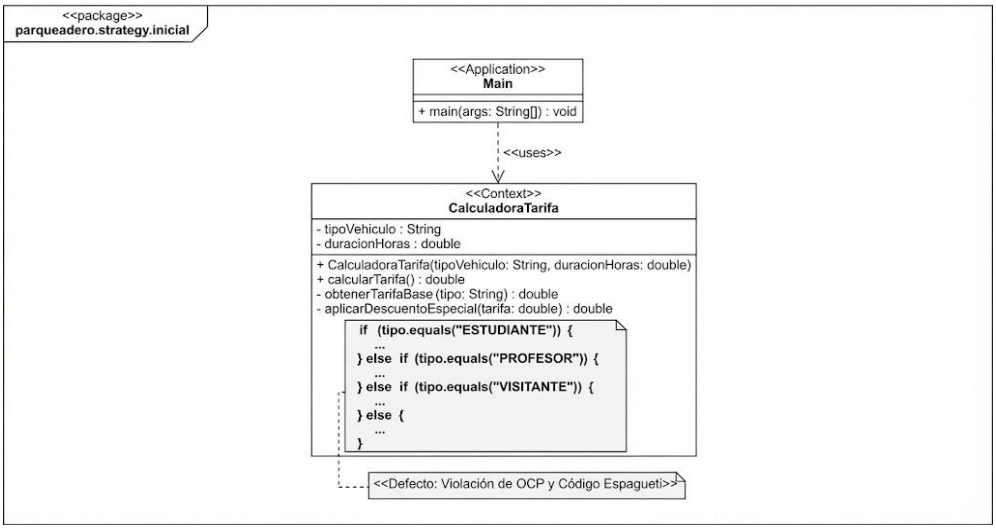
---

3. Modelado y Diagramas UML

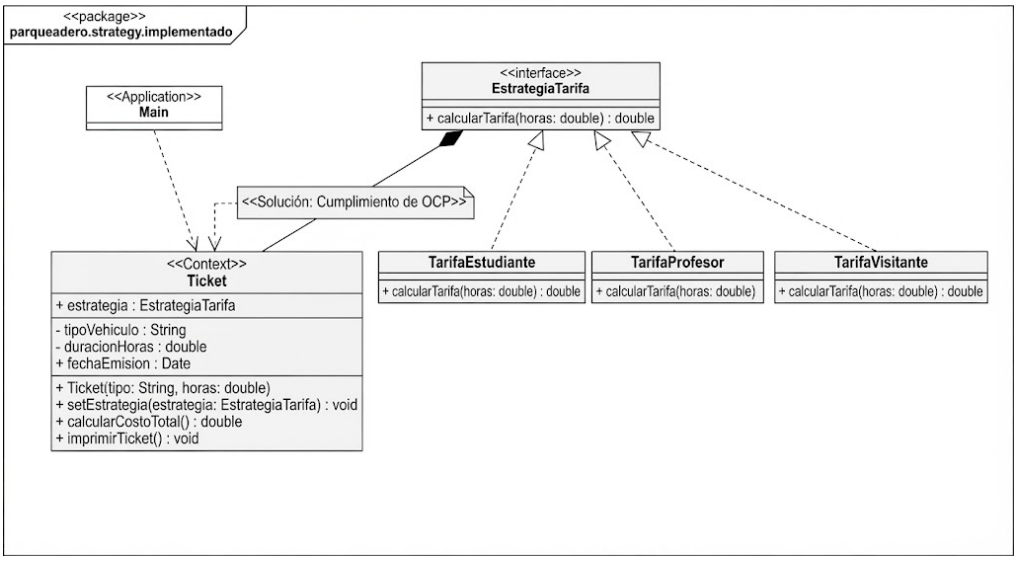
3.1 Patrón Strategy (Cálculo de Tarifas en el Parqueadero)

- **Versión Inicial (Con defecto):** Concentraba la lógica en la clase `CalculadoraTarifa` mediante una sentencia condicional (`switch-case` / `if-else`) gigante que crecía con cada nuevo tipo de usuario, violando el

principio de abierto/cerrado (OCP)

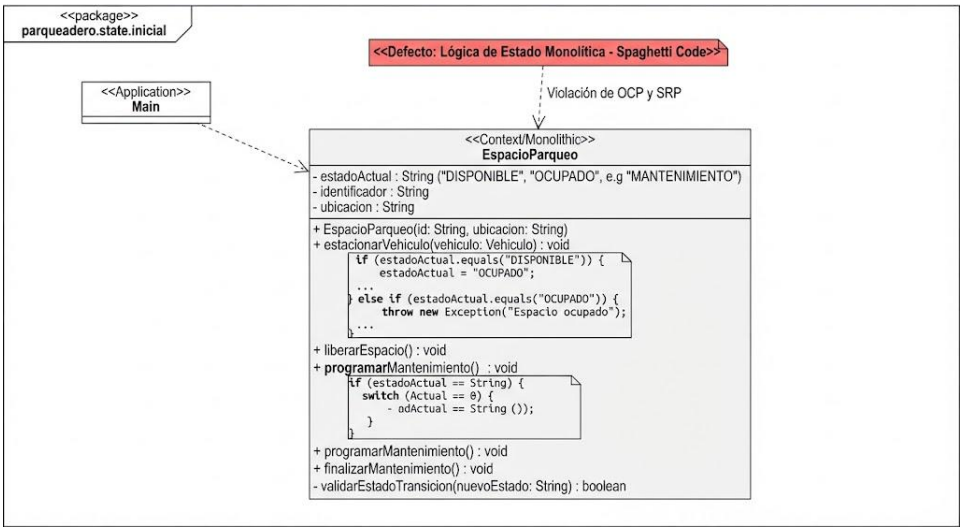


- **Versión Refactorizada:** Introduce la interfaz `EstrategiaTarifa` implementada por clases concretas (`TarifaEstudiante`, `TarifaProfesor`, `TarifaVisitante`), desacoplando el algoritmo de cálculo del objeto `Ticket`

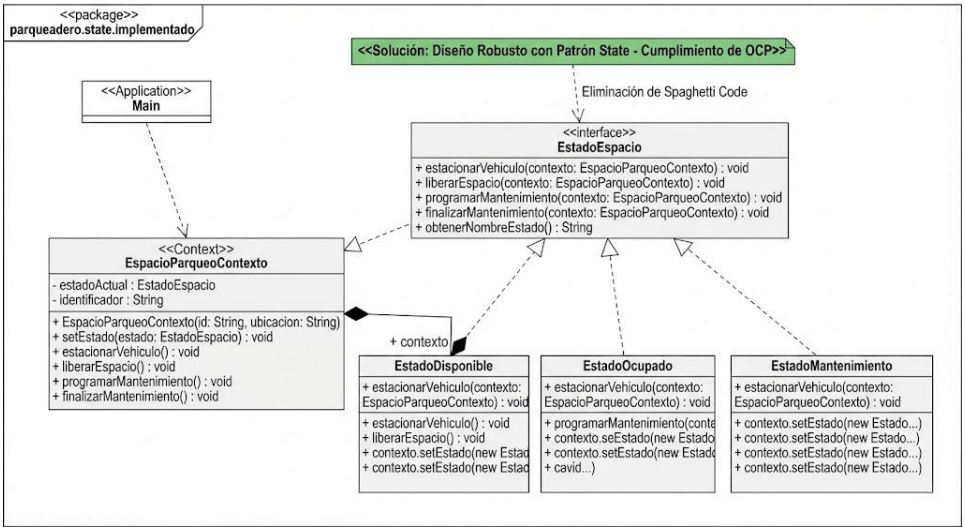


3.2 Patrón State (Gestión de Espacios de Parqueo)

- **Versión Inicial (Con defecto):** La clase `EspacioParqueo` manejaba múltiples banderas de estado y bloques condicionales anidados dentro de cada operación (estacionar, liberar, dar mantenimiento), provocando código rígido y propenso a errores.



- **Versión Refactorizada:** Se implementa el contexto `EspacioParqueoContexto` junto con la interfaz `EstadoEspacio` y los estados concretos (`EstadoDisponible`, `EstadoOcupado`, `EstadoMantenimiento`), encapsulando el comportamiento específico de cada estado



---

4. Banco de Ítems Tipo Saber Pro

Ítem 1 (Strategy - Aplicación de Patrón y Defecto)

Enunciado: Un equipo de desarrollo está construyendo el módulo de facturación para el sistema de un parqueadero. Actualmente, la clase `ProcesadorFactura` calcula el valor a pagar mediante un bloque `switch(tipoVehiculo)` que evalúa si el vehículo es Carro, Moto o Bicicleta, aplicando recargos o descuentos específicos. El cliente solicita

incorporar próximamente tarifas eléctricas y vehículos pesados, lo que obligará a modificar constantemente esta clase y recompilar el sistema.

Para corregir este defecto de diseño garantizando el principio de abierto/cerrado (OCP), ¿cuál de los siguientes patrones de diseño resulta estructuralmente idéntico en su invocación pero apropiado para resolver esta necesidad?

- **A) Strategy**, encapsulando cada algoritmo de tarificación en clases separadas que implementan una interfaz común, permitiendo que el contexto delegue el cálculo sin conocer los detalles de cada tarifa. *(Clave)*
- **B) State**, permitiendo que la clase `ProcesadorFactura` modifique su comportamiento interno cambiando dinámicamente de estado según el tipo de vehículo ingresado.
- **C) Observer**, estableciendo una dependencia de uno a muchos para que los vehículos notifiquen automáticamente al procesador cuando cambie su tarifa.
- **D) Facade**, proveyendo una interfaz unificada para simplificar el subsistema complejo de cálculo de cobros y recargos vehiculares.

Justificación de la clave: El patrón *Strategy* permite intercambiar familias de algoritmos encapsulados (cálculos de tarifas) de forma transparente. Corrige directamente el defecto de las estructuras condicionales masivas (*switch-case*) y cumple con el principio de abierto/cerrado (OCP).
Plausibilidad de los distractores:

- *B: State* comparte la misma estructura de delegación basada en interfaces, pero su intención es la gestión de estados internos y transiciones, no la selección independiente de un algoritmo de cálculo.
- *C: Observer* desacopla emisores de receptores ante eventos, lo cual no resuelve la lógica de selección de algoritmos de tarificación.
- *D: Facade* simplifica el acceso a un subsistema complejo, pero no desacopla algoritmos intercambiables.

Ítem 2 (State - Transiciones y Comportamiento)

Enunciado: En el sistema de control de accesos de un parqueadero automatizado, un espacio físico se modela mediante el patrón *State*. Las operaciones de `estacionarVehiculo()` y `liberarEspacio()` se comportan de manera totalmente diferente según el estado actual (`Disponible`, `Ocupado`, `EnMantenimiento`). Durante la ejecución, cuando un vehículo ingresa a un espacio disponible, el método

correspondiente ejecuta la lógica de ocupación y **ordena internamente al contexto la transición** hacia el estado `Ocupado`.

A partir de este comportamiento, ¿cuál es la característica fundamental que diferencia la implementación de este patrón frente a *Strategy*?

- **A)** En *State*, los objetos de estado suelen conocer al contexto y gestionar las transiciones internas entre ellos, mientras que en *Strategy* el cliente selecciona externamente el algoritmo a utilizar sin que las estrategias cambien por sí mismas. *(Clave)*
- **B)** En *State* las clases concretas no implementan ninguna interfaz común, a diferencia de *Strategy* donde la interfaz es obligatoria para garantizar el polimorfismo.
- **C)** En *State* el comportamiento se define estáticamente en tiempo de compilación, mientras que en *Strategy* se modifica dinámicamente mediante herencia múltiple.
- **D)** En *State* el contexto es completamente pasivo y desconoce los estados existentes, delegando toda la lógica de negocio en una clase controladora centralizada.

Justificación de la clave: Una de las diferencias clave entre ambos patrones radica en quién controla el cambio: en *State*, los estados manejan la evolución y transición del ciclo de vida, en *Strategy*, la selección del algoritmo es externa y desacoplada del contexto operativo.
Plausibilidad de los distractores:

- *B:* Ambos patrones se basan en el uso de interfaces o clases abstractas para garantizar el polimorfismo.
- *C:* Ambos patrones operan y resuelven problemas de flexibilidad en tiempo de ejecución.
- *D:* En *State*, el contexto interactúa estrechamente con los estados, por lo que no es un componente pasivo desconectado.

Ítem 3 (Código de Implementación - Análisis de Defecto y Patrón)

Enunciado: Analice el siguiente fragmento de código perteneciente a la versión inicial del módulo de espacios de parqueo:

```
public class EspacioParqueo {
```

```
    private String estado = "DISPONIBLE"; // "DISPONIBLE", "OCUPADO",
"MANTENIMIENTO"

    public void estacionar() {

        if (estado.equals("DISPONIBLE")) {

            estado = "OCUPADO";

            System.out.println("Vehículo estacionado con éxito.");

        } else if (estado.equals("OCUPADO")) {

            System.out.println("Error: El espacio ya está ocupado.");

        } else if (estado.equals("MANTENIMIENTO")) {

            System.out.println("Error: El espacio está en mantenimiento.");

        }

    }

}
```

Si se refactoriza este diseño aplicando el patrón **State**, ¿cuál es el beneficio principal obtenido en la estructura del código?

- **A)** Se eliminan los bloques condicionales anidados y la verificación por cadenas de texto, distribuyendo el comportamiento específico de cada estado en clases dedicadas que encapsulan las transiciones. *(Clave)*
- **B)** Se transforma el sistema en una arquitectura concurrente donde múltiples hilos pueden acceder al espacio sin bloqueos síncronos.
- **C)** Se optimiza la velocidad de ejecución al evitar el uso de comparaciones lógicas en tiempo de ejecución mediante la compilación estática de estados.
- **D)** Se convierte el objeto en una única entidad inmutable cuyo estado no puede alterarse bajo ninguna circunstancia después de su instanciación.

Justificación de la clave: El patrón *State* elimina la necesidad de grandes estructuras condicionales basadas en atributos de tipo bandera o cadenas de texto, delegando la

responsabilidad del comportamiento y las reglas de transición a clases concretas de estado.

Plausibilidad de los distractores:

- *B:* El patrón soluciona problemas de diseño y mantenibilidad, no de concurrencia multihilo de forma directa.
- *C:* El uso de polimorfismo y clases adicionales introduce despacho dinámico, no una optimización de velocidad por compilación estática.
- *D:* La esencia del patrón *State* es precisamente permitir que el objeto altere dinámicamente su comportamiento al cambiar de estado.
